

下一代 AI 编译器综述

下一代 AI 编译器综述

预测智能体编译器（约 2027–28 与未来五年）：软件栈重塑与软硬件协同设计

持续更新型综述导出

生成日期: 2026-08-03

源仓库: ai-compiler-survey

主叙事: docs/SURVEY.md · 路线图: docs/ROADMAP.md · 软件栈: docs/STACK.md

北极星目标。智能体负责语义搜索、编排与产物综合；编译器负责下降、合法性、测量与回退。硬件协同设计仅通过内核、IR、测试与 profiling 闭环进入——而非自动流片。

调查叙述

最后更新：2026-07-31
配套文献摘要： ../图书/
状态： ../STATUS.md
冲突： 冲突.md · 仓库地图： REPOS.md · 产品： 产品.md

0.1 北极星目标

主要目标：预测下一代智能体编译器（架构+流程将持续到2027-28年和~5年），包括它如何重塑软件堆栈和HW-SW软硬件良好设计 - 而不会陷入通用EDA。

其他一切（论文、GitHub/Gerrit、商业 SKU、论坛、ASIC 使能/拉起研究）都是该预测的证据，而不是其本身的目录。当消息来源不同意时，他们会进入冲突.md，而不是默默地平均。

执行结论。编译正在从固定通道管道+黑盒自动调整转向混合LLM-编译器循环。根据经验，获胜模式是：

**** 智能体负责语义搜索、编排和工件合成。编译器负责降低、合法性、测量和回退路径。 ****

智能体重塑控制面的程度超过取代数据面的程度。第四项工作 — 加速器使能/拉起 / 软硬件良好设计反馈 on sim+silicon — 现在是 A 级证据（TritonX、KernelEvolve），仍然以内核/IR/判定机为中心。请参阅 [路线图.md](#)、[堆栈.md](#)、[§5](#)、[§6](#)、[§4](#)。

1.现在的趋势是什么？(Q1)

1.1 两个堆栈汇聚

堆栈	提问即回答	成熟度
人工智能编译器	我们如何有效地将神经图降低到 GPUs/TPU？	成熟基板（TVM/XLA/MLIR/ Triton）；仍然支离破碎
编译器人工智能	我们如何选择传递、编写启发式/内核、使用反馈？	2023-2026 年快速增长；混合动力系统占主导地位

“下一代”并不是“扔掉 LLVM/MLIR”。它使这些堆栈智能可寻址：结构化摘要、有界候选空间、工具 API 和奖励判定预示机。

1.2 时代时间表

时代	标签	发生了什么变化
2018–2022	深度学习编译器成熟	TVM/Ansor、XLA、发光；MLIR因TF/XLA需求而诞生；成本模型自动调整
2020–2023	RL 编译器的 ML	Autophase、CompilerGym、MLGO 内联/regalloc；神经政策难以实施
2023–2024	LLM进入IR	法学硕士合格名单；Meta LLM编译器基础模型；编译器反馈循环
2025–2026	智能体混合时代	多智能体调谐器、验证器-RL、启发式综合、Triton 内核智能体、生成编译、供应商 CompileIQ / GEAK

1.3 六大活跃趋势（详）

趋势 A — 混合指导，而不是 LLM 作为编译器

直接 LLM IR 重写反复脆弱。mlir 智能体报告前沿模型在 IR 变换上的得分低于同一性。成功的系统限制了法学硕士：

- 智能体编译：LLM 仅发出咨询元数据；模板+检查+基准测试承认CUDA。
- **HintPilot**：LLM 插入编译器验证的编译指示/属性，而不是任意重写。
- **Meta LLM编译器/Compiler-R1**：LLM提出优化遍历序列；opt 应用它们。

趋势 B — 从 RL 健身房到 LLM 智能体

CompilerGym 公开了 LLVM 作为 OpenAI Gym 环境的通行证（Autophase 功能、IR 指令计数奖励）。这降低了强化学习研究人员的障碍，但不同编译器版本的策略往往不透明且脆弱。

2024-26 年将转向：

1. 基础模型 在 IR/asm 上进行预训练（Meta LLM 编译器：546B 令牌）。
2. 使用工具的智能体经过SFT+RL (Compiler-R1) 训练。
3. 仅限推理的多智能体调谐器，避免繁重的离线训练 (AutoPass)。
4. 启发式程序综合，以普通 C++ (Magellan) 的形式实现，恢复了编译器中的神经网络所缺乏的可部署性。

趋势 C — MLIR + Triton 作为默认基材

今天的实用 GenAI 路径：text PyTorch → Dynamo/Inductor → Triton → PTX/CUDA (or CUDA Tile IR)并行可移植路径：text Framework → StableHLO → XLA or IREE → device backends即使产品品牌不同（OpenXLA、Triton 内部结构、供应商 AI 编译器、CIRCT），MLIR 仍然是共享的工程基础。行业评论（Modular/Lattner）指出碎片化，并且开放的 MLIR AI 方言并不总是与 LLM 推理的 CUDA 峰值相匹配，因此出现了 Triton、CUTLASS、FlashAttention 以及现在的 CUDA Tile / CompileIQ。

趋势D——内核智能体走向工业化

编写 GPU 内核是新模型架构和非 NVIDIA 可移植性的瓶颈。

- **KernelBench (ICML 2025)**: 250 个 PyTorch 任务；指标“fast_p”=正确并且比基线更快。前沿模型的一次性结果通常仍低于 20%；迭代细化有帮助。
- **KernelBench-X**: 细化提高了正确性，但可能损害平均加速；许多正确的内核仍然输给了 eager；核聚变任务依然艰巨。
- **GEAK (AMD)**: Instinct 上 Triton 的生成器/评估器/反射器/优化器循环；据报道，套件的执行准确度高达约 63%，加速速度高达约 2.59 倍。
- **元KernelLLM**: PyTorch→Triton专用的8B型号； Pass@k 与 KernelBench-Triton 上更大的通用型号相比具有竞争力。
- **智能体编译**: 用于变压器推理图的编译器限制的 CUDA 专业化。

趋势 E — 验证进入循环

正确性策略，从最弱到最强（局部）：

- 1.编译+单元测试/LLM生成的测试（ACCLAIM）。 2. 数值检查与参考路径（智能体编译）。 3. 往返/重新编译检查（LLM 编译器反汇编）。 4. 形式语义等价（LLM-VeriOpt 奖励中的 Alive2）。

正式报道仍然主要是局部窥视孔/IR。全程序 GPU 竞赛和 FP 不确定性缺乏同样强大的判定预言机。

趋势 F — 编译器拓宽其对象

- 生成式编译：密封器完成部分 Rust，因此编译器可以在 **LLM 解码期间进行诊断，而不仅仅是在文件完成之后。
- **Compiler.next**: 将 FMware（提示、智能体、自由参数）视为 SE 3.0 的多目标搜索/编译问题。
- 人类 **Claude C** 编译器：智能体团队 构建一个能够构建 *Linux* 内核的编译器 (~100kLoC Rust) - 相邻但重要：智能体作为编译器工程师，而不仅仅是编译时优化器。
- **Magellan / AlphaEvolve**: 智能体重写生产编译器内的启发式 C++（Google 报告生产内联使用情况和 XLA 实验）。

1.4 场馆地图

社区	看什么
CGO / CC / ASPLOS / PLDI / MLSys	系统+编译器
NeurIPS / ICML (+ C4ML)	方法 + KernelBench / 推理编译器 / Compiler-R1
ACL 调查结果	HintPilot 式 SE/NLP 交叉
LLVM 话语（LLVM ♥ ML 研讨会）	MLGO、Magellan、智能体公关评论
供应商博客	NVIDIA CUDA Tile/CompileIQ、AMD GEAK、Meta KernelLLM/LLM 编译器、DeepMind AlphaEvolve、模块化

1b.传统AI编译VS跟随趋势

本节将经典 AI/DL 编译器堆栈（图形捕获 → 固定/自动调整降低 → 内核 → 运行时）与上面总结的 2024-2026 年智能体 / LLM 混合趋势进行比较。重点不在于“旧的坏/新的好”，而是双方在哪里获胜以及混合体应该保留什么。

这里的“传统”是什么意思

层	传统方法	代表系统
前端	跟踪/导出图表； op 融合规则	TorchDynamo/Inductor、TF/JAX → HLO/StableHLO
中端	方言下放+手写通行证	MLIR、XLA、TVM 继电器/TIR
计划/自动调谐	成本模型+进化/模板搜索	AutoTVM、Ansor、MetaSchedule
启发式	专家 C++ 阈值（内联、regalloc.....）	LLVM -03、MLGO 神经顾问（仍在树中）
内核	库 + 专家 CUDA/Triton/CUTLASS	cuDNN、FlashAttention、手部 Triton
正确性	编译器语义+单元/黄金测试	确定性传递；规模有限的正式
服务	独立的运行时堆栈	vLLM、TensorRT-LLM、CUDA 图表

逐维度比较

尺寸	传统人工智能编译——优点	传统——缺点	跟随趋势（法学硕士/智能体混合）——优点	趋势——缺点/新风险
正确性模型	数十年的通行工程经验；常见路径的错误编译率较低	错过意图级别的选择；GPU/FP全程序难以证明	Gates（Alive2、测试、模板）可以将编译器作为事实来源	不受约束的LLM重写是不安全的；判定天气预报机仍为本地
搜索/改编	经过足够的试验，自动调谐发现了强大的时间表	样本效率低下；跨形状/硬件/版本的弱迁移	语言先验+MCTS/智能体切样；多层次的创造力	代币成本、不确定性、难以重放的痕迹
启发式维护	人类可读、可审查的C++	专家成本高；陈旧的vs新的硬件/型号	Magellan式可部署启发式综合；MLGO博学顾问	智能体代码的所有权、回归、供应链
内核专业化	专家投资时实现巅峰业绩	无法扩展到每个新的操作/硬件（尤其是非NVIDIA）	GEAK/KernelLLM/智能体编译自动探索-测量循环	正确≠快速（KernelBench-X）；融合仍然困难
生产准备	PyTorch/XLA/LLVM/CUDA中的默认路径	堆栈碎片；Peak通常需要供应商库	供应商举措（CompileIQ、GEAK、Magellan产品内联）显示出吸引力	很少有智能体循环是默认编译标志
便携性	StableHLO / MLIR瞄准框架↔编译器可移植性	方言和Triton/Tile/PTX仍然是孤立的原则上智能体可以跨AMD/NVIDIA重定向内核	没有跨堆栈的标准智能体IR/工具合约	
编译对象	图 → 二进制/内核	不包括提示、智能体、FMware旋钮	Compiler.next / 生成式编译拓宽对象	早期视力；质量门不成熟
人类角色	专家编写通道/内核；审核周期长	稀缺专业知识的瓶颈	智能体草案；人类审查/编排	审核能力&安全成瓶颈
成本模型	编译时CPU + 自动调整GPU小时（已知）	为大空间自动调整墙壁	可以减少搜索试验	添加LLM\$/延迟；预算缺乏标准化
可解释性	通行证清单和时间表可检查	为什么启发式触发仍然不透明？自然语言原理+可能的痕迹	基本原理可能与真正的因果选择不匹配	

保持传统的优点（何时不强迫智能体）

1. 稳定、可回归的默认设置 — -03、Inductor、XLA 管道经过数百万次构建的实战测试。
2. 确定性 CI — 相同的 IR 输入 → 相同的二进制输出（模已知的非确定性）；除非仔细缓存，否则智能体循环会打破这一点。
3. 所有权清晰——通证有维护者；智能体的轨迹通常不会。
4. 峰值库内核 — FlashAttention/CUTLASS 仍然主导着许多热路径，而循环中没有 LLM。
5. 形式/工具成熟度 — Alive2、LLVM lit、FileCheck 和供应商验证器假设经典工件。

跟随趋势的优点（当智能体赢得生存时）

- 1. 按程序/按模型专业化超出全局启发式允许的范围（传递顺序、提示、CompileIQ 旋钮）。
- 2. 更快地探索具有语言先验的内核和调度空间（推理编译器，GEAK）。
- 3. 缩小意图差距 - 当测试限制它们时，算法级重写编译器无法发明（ACCLAIM）。
- 4. 编译器工程杠杆 — 综合启发式/传递 (Magellan) 或使用领域工具审查 opt PR（LLVM Discourse 智能体审查）。
- 5. 新表面 — 解码中诊断（生成编译）； FMware 多目标编译（Compiler.next）。

合成：保留底物，改变控制面``text

Traditional strengths to preserve deterministic lowering · library kernels · CI regressability · formal local checks

Trend strengths to adopt advisory search · multi-智能体 orchestration · 启发式综合 · profile/verifier feedback

Anti-pattern replace opt/Inductor with unconstrained LLM codegen and hope tests catch it ``建议：将传统人工智能编译器视为数据面，将智能体/LLM方法视为带有承认/回退路径的可选控制面。通过 (a) 生产默认采用、(b) 可重播跟踪、(c) 判定预测机强度来衡量成功，而不是仅通过微基准加速来衡量。

2.智能体如何助力AI编译？(Q2)

2.1 机制（为什么有效）

- 1. 样本效率 - 语言先验提出合理的时间表/通道，而不是盲目的进化样本（推理编译器与 MetaSchedule 式搜索）。
- 2. 正确性包络 - 对提示、模板、经过验证的 IR 或优化遍历序列的干预进行约束，避免不受约束的代码生成失败。
- 3. 多层次创造力 - 编译器错过了目的级别的重写；法学硕士可以在测试门验收时重构算法 (ACCLAIM)。
- 4. 编译器工程本身 - 智能体编写可维护的启发式/通道（Magellan、mlir 智能体），而不是在“opt”内传送不透明的神经网络。
- 5. 反馈致密化 - 分析器、编译器诊断、Alive2 和快速反馈将稀疏奖励转变为迭代细化循环。

2.2 闭环（规范）

请参阅[分类.md](#)。变种：

变体	扭转
生成编译	通过密封器在解码过程中提供反馈
推理编译器	LLM提案扩大MCTS节点
Magellan	C++ 启发式的进化变异 + Vizier 自动调谐
ACCLAIM	多级预算智能体+测试智能体
GEAK	GPU 上的反射式生成-评估-反射-优化
CompileIQ	编译器内部旋钮的进化搜索（非源代码）

2.3 智能体不应该拥有什么

- 未经检查的红外/装配发射作为生产代码，没有接纳门控。
- 无需黄金或统计检查即可静默替换数字内核。
- 没有可靠工具调用的编排（ACCLAIM：开放模型在代码质量之前因格式错误的工具调用而失败）。

3. 智能体能否重塑编译过程？(Q3)

简短回答：对于控制面是的；（到目前为止）还没有大规模替代确定性降低的方案。

这些重塑声明是S5未来预测的证据。下面的硬性限制以及 S4 中的差距是预测未来的障碍**。分层存储库/产品证据：REPOS.md、产品.md。

3.1 旧→新流程图

旧流程	智能体重塑过程	证据
修复了“-O2/-O3/-Oz”管道	按项目通过 LLM/RL 政策进行搜索	Compiler-R1、AwareCompiler、AutoPass、LLM 编译器
C++ 中的手动调整启发式	智能体合成/进化启发式代码	Magellan（LLVM 内联/regalloc；XLA）
黑盒进化自动调谐	情境感知法学硕士提案+结构化搜索	推理编译器（TVM + MCTS）
编译→失败→人工修复	部分生成期间的编译器反馈	生成编译
专家写Triton/CUDA	智能体生成-评估-反映-优化	GEAK、KernelBench 智能体、KernelLLM
单IR电平优化	多智能体选择抽象级别	ACCLAIM
编译源→二进制	编译意图/FMware 旋钮	Compiler.next（愿景）
通用 NVCC 启发法	每内核进化编译器控制	NVIDIA CompileIQ
人类编写编译器	智能体团队实现 C 编译器	人择CCC（工程实验）

3.2 硬限制

1. 由法学硕士进行的直接 IR 转换可能不如身份；搜索/启发式综合更可靠地获胜（mlir智能体）。
2. KernelBench-X：迭代精炼可以提高通过率，但平均加速比会下降；大约 46% 的正确内核在其设置中仍然比 eager 慢。
3. 开放模型的工具调用质量可以在代码技能破坏之前破坏多智能体编译器（ACCLAIM）。
4. 形式验证器涵盖局部等价物；他们还没有支持端到端 GPU 服务堆栈。
5. 智能体编译循环（令牌、不确定性、硬件噪声）的成本和可重复性保持开放（Compiler.next 号召性用语）。

3.3 实用架构推荐

将智能体形编译器设计为：
text bounded candidate spaces + structured IR / region summaries + advisory LLM + deterministic materialize + empirical and/or formal admit + 回退路径将离线编译器工程智能体（Magellan 式信息式综合）与在线编译时智能体（HintPilot / 智能体Compile / CompileIQ）分开。

4. 遗漏/未涵盖哪些内容？（第四季度）

下面的差距不是一个单独的“愿望清单”——它们是\$5预测的未来的障碍（智能体可寻址数据面、三个智能体工作、一流的工件、经典默认值，直到 CI 证明智能体）。覆盖范围参差不齐。每个差距都说明了存在什么，缺少什么，为什么它阻碍了未来，以及“完成”可能是什么样子**。摘要：[../图书/](#)。证据图（A级证据/B/C）：[REPOS.md](#)，[产品.md](#)。

差距图（优先快照）

#	差距	现在的严重性	谁最先感受到
4.1	端到端生产证据	高	框架/编译器厂商
4.2	规模正确性	高	任何发布内核或 IR 重写的人
4.3	智能体循环的成本和可重复性	高	CI/发布工程
4.4	4.4跨堆栈互操作性	高	多后端平台
4.5	4.5硬件原生智能体接口	中-高	智能体+编译工具构建者
4.6	FMware / 智能体-app 编译	中等	LLM-app / SE 3.0 平台
4.7	4.7编译器培训数据	中-高	基础模型构建者
4.8	人机循环编译器工程	中等	LLVM/XLA 维护者组织
4.9	4.9安全/供应链	中等（上升）	产品基础设施和开源
4.10	4.10统一基准	高	研究+行业比较

4.1 端到端生产证据

存在什么。许多论文报告了内核级、通过级或微基准级胜利：IR 指令计数上的 Compiler-R1；PolyBench 上的 HintPilot；所选服务内核上的推理编译器；模型或内核套件上的智能体编译/GEAK；Magellan 谈论声称生产内联使用；NVIDIA CompileIQ 在已经很热门的 GEMM/注意力上的报价≤15%。

缺少什么。很少有智能体或 LLM 方法是任意用户程序的 PyTorch、Inductor、OpenXLA/XLA 或 LLVM 主干中的默认路径。胜利往往是：

- 选择加入标志或研究分叉；
- 在精选内核上进行评估，而不是在完整的训练/服务堆栈上进行评估；
- 难以归因（自定义 GEMV / CUDA 图表 / KV 缓存与“LLM 指导”）。

为什么它会阻碍进展。如果没有默认路径证据，组织就无法证明始终在线的智能体编译成本或审查负担的合理性。调查声称存在夸大准备情况的风险。

完成看起来像。主要框架上的公共 A/B（例如 Inductor 默认与智能体专用解码路径），具有回归套件、推出标志和多月稳定性 - 类似于 MLGO 在部署后报告持续 QPS 增益的方式。

4.2 规模正确性

存在什么。

甲骨文	实力	弱点
编译器应用通道 (LLM 编译器、Compiler-R1)	继承的通行证语义	无法安全地发明新的变换
单元/LLM 生成的测试 (ACCLAIM)	发现许多功能错误	低于近似值；片状测试
数值检查与参考 (智能体编译)	适合金黄色的玉米粒	宽容游戏；训练与推断数值
Alive2 / 正式 (LLM-VeriOpt)	强大的本地 IR 等效性	窥视孔/本地；不是GPU并发
往返拆卸检查	部分信任信号	精确匹配率仍然较低

缺少什么。 Oracle 的用途： - 多级智能体重写后的全程序语义对等； - 浮点合约（重新关联、TF32/BF16、降阶）； - GPU 竞赛、异步副本、扭曲发散和内存模型错误； - 服务级别行为等效（采样、KV 布局、推测解码）。

为什么它会阻碍进展。 KernelBench-X 已经表明正确的内核可能会很慢；双重风险是“快速但微妙的错误”。仅仅停留在窥视孔的形式化方法使得最高价值的智能体行为（融合、调度、算法重写）得不到充分的管理。

完成的样子。 分层准入政策：可能的地方正式→形状网格上的差异测试→统计服务检查→分阶段推出。 Triton/CUDA Tile IR 共享开放的判定主机，而不仅仅是 LLVM IR。

4.3 智能体编译循环的成本和可重复性

存在什么。 快速反馈（~10 倍迭代与完整 IR 内环）； CompileIQ 生成便携式高级控制文件；有些论文确定种子或报告样本预算； Compiler.next 显式调用可重现的意图编译和共享跟踪。

缺少什么。

- 标准化成本模型（代币+编译分钟+GPU小时/%增益）；
- 跨模型版本智能体决策的确定性重播；
- 可缓存编译跟踪（IR 哈希、硬件、编译器版本、智能体策略）；
- 用于片状加速的 CI 策略（硬件噪声、DVFS、争用）。

为什么它会阻碍进度。 花费 N 美元的 LLM 调用且无法重现的编译不是一个编译器功能，而是一个实验。发布工程将拒绝它。

完成看起来像。 跟踪格式+内容寻址缓存（优化遍历序列/提示/ACF/内核工件）签入构建系统；预算 SLO；升级智能体模型时进行黄金回放测试。

4.4 跨堆栈互操作性

存在什么。 StableHLO 用于框架↔编译器图； MLIR 作为共享想法； Triton 作为事实上的 GPU DSL；供应商桥（实验性 Triton→Tile IR）。

缺少什么。智能实体可见状态的可移植合约：

概念	需要跨堆栈	今天
区域/融合候选	通用架构	每篇论文特设
约束集（形状、别名、硬件）	共享词汇	嵌入提示
动作空间（通过、提示、时间表、模板）	枚举和版本控制	每个系统关闭
奖励/承认结果	结构化反馈	自由文本日志
神器身份	可哈希输出	经常丢失

方言、Triton、CUDA Tile IR、PTX 和 LLVM IR 仍然处于孤立状态；在一个堆栈上调整的智能体很少会转移。

为什么它会阻碍进步。每个新的智能体都会重新实现胶水。多后端公司无法分享经验。

完成看起来像。一个“智能体编译接口”RFC（可能在 StableHLO 或 MLIR 上）定义摘要、操作和承认记录 - 本质上类似于 PJRT 标准化运行时插件的方式。

4.5 硬件原生智能体接口

存在什么。mlir智能体：结构红外指纹、知识图、MCP工具套件；Compiler-R1 工具调用（instrcount 等）；GEAK 硬件反馈回路；NVCC/PTXAS 内部的 CompileIQ 搜索空间(NVIDIA/XProtect17X)；Archer 中的 SCM 端工具 API（“verify”、“diffest”、“trans”、工作流程）绑定到本地“llvm-project”结帐；Gerrit AI 智能体 提供用于 UI 内聊天的 API（通用法学硕士，除非扩展）。

缺少什么。标准，而不是演示：

- 通行证效果的共同指纹/出处；
- 供应商中立的 MCP（或同等）工具模式，用于“编译/验证/基准测试”；
- 用于大规模执行智能体建议的内核的安全沙箱；
- 硬件计数器对智能体的一流曝光，无需抓取分析器文本。

为什么它会阻碍进展。没有结构化接口的智能体会退回到脆弱的提示和祈祷“opt”标准输出。

完成的样子。LLVM/MLIR/Triton 工具 API，智能体可以使用类型化 I/O 调用；指纹识别作为支持分析；工具服务器的一致性测试。

4.6 FMware /智能体-应用程序编译

存在什么。Compiler.next 愿景：多目标质量门下的编译提示、智能体拓扑和自由参数；生成编译将编译器耦合到智能体编码中；工业智能体利用（Claude C编译器）压力测试构建。

缺少什么。FMware DL 编译器的成熟类似物：

- 提示/工具图的稳定IR；
- 当质量阈值未达标时，编译失败关闭；
- “提示编译器”和经典模型编译器之间的互操作性；

- 社区学习的共享痕迹 (Compiler.next 号召性用语#10)。

为什么它阻碍了进展。LLM 应用程序仍然通过手工和民间传说进行调整，而 DL 图享受了数十年的编译器投资。

完成看起来像。可复制的 FMware 编译管道具有黄金标签、成本/延迟/质量 Pareto 前沿以及阻止回归的 CI——与当今模型动物园运送编译工件的方式类似。

4.7 编译器的训练数据

存在什么。Meta LLM 编译器：LLVM-IR/汇编的 546B 令牌 + 编译器仿真指令数据；KernelLLM：~25k torch↔Triton 对 (KernelBook)；Compiler-R1 推理数据集 (~19.6k)；CompilerGym 工作负载；MLGO 训练语料库 (通常是内部的)。

缺少什么。大型、开放、版本化语料库：

- MLIR 方言 (linalg、scf、affine、供应商方言)；
- Triton / Tile IR / PTX 与时间表和性能标签配对；
- StableHLO 图表显示结果降低；
- 批评者/验证者的负面数据 (编译失败、编译错误、内核缓慢)。

为什么它会阻碍进展。如果没有数据，选择器/生成器模型就会保持以 LLVM 为中心或过度拟合 KernelBench。现代 AI IR 的 Meta LLM 编译器的后续版本仍然很少。

完成的样子。公开的“编译器的 ImageNet”2.0：多 IR、多硬件，已获得商业研究许可 - 建立在 CompilerGym 最初的雄心之上。

4.8 人机循环编译器工程

存在什么。Magellan 产生可审查的 C++ 启发式；Archer (纸, [GitHub](#)) 智能体评论 LLVM GitHub PRs 与 Alive2/LLUBI 证据门；LLVM Discourse 线程报告类似的智能体 PR 审核体验；Gerrit 托管大型组织变更工作流程中使用的通用人工智能审查插件 (ai 代码审查、评论人工智能、GerritForge)；Anthropic CCC 强调安全带；拉特纳评论强调测试是真正的产品。有关 SCM 映射，请参阅 [REPOS.md](#)。

缺少什么。处理答案，而不仅仅是模型：

- 六个月后谁拥有智能体书写的通行证？
- 当作者是智能体时，代码审查有何不同？
- 硬件刷新后，人类什么时候会覆盖智能体启发法？
- 如何在不崩溃的情况下混合智能体草案+专家定稿？

为什么它会阻碍进展。如果智能体工件无人拥有，那么生产率的提高就会逆转为维护债务。如果没有领域工具，通用 SWE 智能体在 opt 审查方面表现不佳，因此 HITL 设计具有研究价值。

完成看起来像。记录的工作流程：智能体建议 → 自动化判定机门 → 人工审查清单 → 所有权分配 → 回归语料库更新。审查时间与逃逸错误的指标。

4.9 安全/供应链

存在什么。 公开讨论稀疏：人性作者指出对未经验证的部署代码感到不安；经典编译器供应链问题（信任 opt、二进制来源）；在本次调查的目录中几乎没有关于对抗性智能体内核的专门论文。

缺少什么。

- 恶意或有缺陷的智能体内核的威胁模型（无声的错误答案、侧通道、通过病态时间表的 DoS）；
- 智能体生成的启发式/内核的来源签名；
- 用于编译时执行不受信任提案的沙盒策略；
- SBOM 类记录：哪个模型、提示、工具和接纳门控产生了工件。

为什么它会阻碍进展。 随着 Magellan/GEAK/CompileIQ 类型的工件进入生产二进制文件，它们成为可信计算基础的一部分。

完成看起来像。 智能体承认的工件需要安全审查；可重复的构建；在线智能体的许可名单；用于内核智能体的红队套件。

4.10 统一基准

存在什么（碎片）。

套房	措施	盲点
CompilerGym / Autophase 型	LLVM IR 尺寸/通过 RL	不是 GPU 服务
PolyBench / HumanEval-CPP (+ HintPilot)	CPU 运行时与提示	不是 IR 智能体或内核
KernelBench / KernelBench-X / TritonBench	GPU 内核正确+快速	不是完整的服务图表
纸张专用服务内核	注意/MoE/MLP 切片	论文之间不具有可比性
供应商内部套件	生产真相	关闭

缺少什么。 共享梯子：

1. CPU IR选项（大小+运行时间）， 2.单内核（CUDA/Triton/Tile）， 3.融合区域，
2. 完整的 LLM 服务图（预填充/解码、批量、长上下文），
3. 多目标（延迟、吞吐量、内存、质量），

具有固定的硬件配置文件、参考坞和排行榜，可区分正确性、性能和编译成本。 为什么它阻碍了进展。 跨论文“加速”比较目前几乎没有意义；该领域无法判断选择器/生成器方法是否正在改善同样的问题。

完成看起来像。 社区基准联盟（LLVM ♥ ML + MLSys + 供应商轨道）发布版本化任务，并禁止在没有完整阶梯的情况下挑选单内核头条新闻。

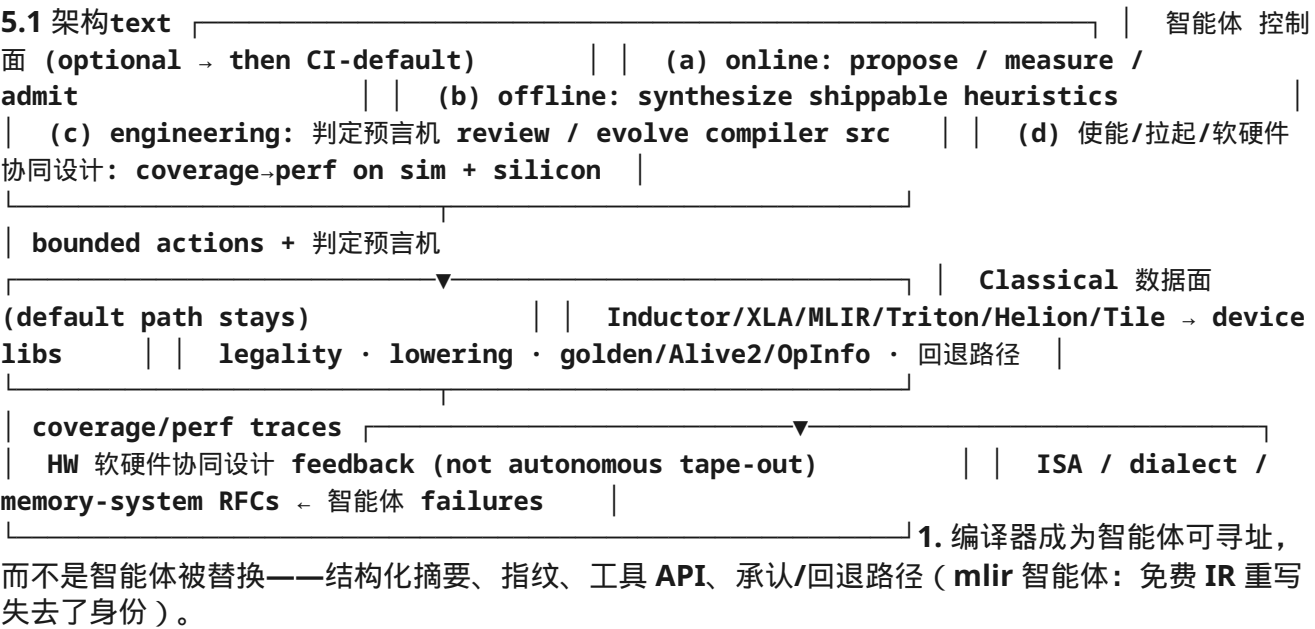
主题	近期工作	取决于间隙
准入及回退路径标准	混合管道规范	4.1、4.2、4.3
智能体编译IR/工具	RFC + MCP 架构	4.4, 4.5
开放多 IR 语料库	MLIR/Triton/Tile 数据集	4.7、4.10
服务级别预测机	差异测试+统计检查	4.2、4.10
出处和 HITL	所有权+签名工作流程	4.8, 4.9
FMware编译MVP	质量门控提示/智能体搜索	4.6, 4.3

针对采用此的组织后续问题

1. 在线编译时智能体（HintPilot/智能体Compile/CompileIQ）与离线启发式综合（Magellan）——哪个符合您的发布模型？
2. 判定预测机的正确性是什么——Alive2、golden kernels 或serving A/B——以及谁拥有假阴性？
3. 哪个IR是智能体合约——LLVM IR、MLIR方言、Triton、StableHLO、CUDA Tile IR？
3. 如何跨编译器和模型升级缓存和回归智能体编译跟踪？
4. 为了获得中值 X% 的胜利，您将花费的最大 \$/build 或 tokens/build 是多少？
5. 合并后每个智能体承认的工件的指定维护者是谁？

5. 未来预测（下一代是什么样子）

~2027–2028 的可证伪草图，以 [冲突.md](#) 中的冲突为条件。



1. 四个智能体工作坚持：(a) 在线特化，(b) 离线启发式/传递综合，(c) 编译器工程和审查，(d) 加速器使能/拉起/软硬件设计反馈。
2. 新的一流工件：ACFs、进化的 C++ 启发式、经过验证的内核、优化内存、使能/拉起语料库、可重播跟踪。

3. 默认保持经典，直到智能体在 CI 的分布上获胜；热门内核、大小关键的应用程序和新 ASICs 首先采用。
4. 不会很快发生：不受约束的LLM取代opt/Inductor而不带判定预测机（C6）；通过编译器智能体（C10）实现自主芯片流片。

5.2 智能体如何改变未来（流程）

遗产	智能体改变未来	信心
专家手写启发法	离线智能体提出可审查的 C++/MLGO 功能	高（Magellan/MLGO 两条运输路径 — 冲突 C1）
自动调谐黑匣子	VCS 中的版本化搜索工件（ACF、跟踪）	中高（CompileIQ 设计）
稀缺内核专家	带有分析器的多智能体内核循环判定机	中（供应商博客强大；KernelBench-X 上限 — C2）
仅限人类选择 PR 审查	编译器 - 决定机关于 PRs/Changes 的智能体	中（Archer；通用锻造 AI 还不够 - C7）
单 DSL (CUDA) 专业知识	多 DSL 智能体技能 (Triton/Tile/CuTe/HIP)	中（TRT-LLM 智能体 PR；GEAK v3 — C4）

5.3 什么会证伪这个预测

- 主要的人工智能堆栈提供默认降低，没有经典的承认/回退路径和持续的正确性。
- Magellan 级合成和 MLGO 神经顾问均从生产中消失（两种路径均不存在）。
- 内核智能体永远处于高原状态，渴望融合重度套件，没有工业解决方法（永远仅限库）。

5.4 近期信号调节草图

来自 Magellan LLVM 开发会议幻灯片 ([文献摘要](#))、ACCLAIM ([arXiv: 2604.04238](#)、[文献摘要](#)) 和 HW-软硬件良好设计智能体：

信号	\$S\$ 的含义	观看
Magellan OSS 通过 OpenEvolve + OSS 型号	离线作业 (b) 可以在 Google 之外复制	公共配方/llvm 补丁 (C1)
Magellan XLA 自动分片/图形重写绿地	离线智能体在人类专业知识匮乏的情况下发明启发式方法端到端 XLA 管道评估（幻灯片：WIP）	
ACCLAIM多级编译器↔LLM合作(代码)	在线作业 (a) 是跨级别编排+测试准入，而不是IR替代	工具调用质量；GPU/服务端口
TritonX 覆盖 MTIA + 未来设备 SIM	Job (d)使能/拉起/软硬件良好设计进入智能体编译器	第二供应商重现 (C9)
KernelEvolve 异构 NVIDIA/AMD/MTIA 性能智能体	生产多硬件控制面	公共跟踪与 KernelBench-X (C2)
Ascend 基于编译器 Triton 诊断	非 CUDA NPU 需要 IR/pass 升级，而不是 CUDA 预训练的猜测	等级制度消融
Helion + CompileIQ ACF路径	DSL基板智能体专业化	C4 与 Tile/CuTe

5.5 五年愿景和路线图指针

详细里程碑：[路线图.md](#)（Horizon A 2027–28，Horizon B ~2029–31）。执行 5 年赌注（仍然以智能体 **ic** 编译器为中心）：经典数据面仍然存在；智能体控制面成为组织在 O（操作 × 设备 × 代数）中生存的方式。硬件软硬件良好设计表现为 **sim/silicon** 反馈到 **IR/ISA/方言**，而不是自主流片（**C10**）。新工件（ACF、启发式、记忆、使能/拉起语料库）位于 VCS 中的二进制文件旁边。

5.6 堆栈整形指针

逐层SW + 软硬件良好设计图：[Stack.md](#)。索赔 ID：[索赔.md](#)。

6. 冲突（指针）

搜索结果经常不一致（供应商标题与文档、Magellan 与 MLGO、免费重写与仅咨询、Triton 与 Tile、在线与离线智能体、覆盖率与峰值使能/拉起、编译器软硬件良好设计与自主 EDA）。不要将这些瓦解成错误的共识。

→ 完整文章：[冲突.md](#)（C1–C10）。

§5中使用的工作姿势：混合控制/数据面；Magellan 和 MLGO 作为平行投注；折扣单数加速；限制行动，直到判定机证明可以自由重写；将通用 SCM AI 降级为 C 级证据；通过覆盖 → 性能智能体阶梯（**C9**）进行软硬件良好设计，而不是自主芯片设计（**C10**）。

7. 如何阅读此存储库

1. 浏览§0.1北极星目标，§5未来预测，然后[路线图.md](#) / [堆栈.md](#)。
2. 检查[侵害.md](#)；当两个来源不一致时，请阅读 [冲突.md](#)。
3. 对于具体系统使用[系统.md](#)。
4. 使用 [REPOS.md](#) / [产品.md](#) 作为 A 级证据/B/C 预测证据，而不是伪造/SKU 目录。
5. 使用 [../图书/INDEX.md](#)（首选★ — ACCLAIM、Magellan、TritonX、KernelEvolve、Kernel*、CompileIQ、Archer）。
6. 通过 [工作流程.md](#) 做出贡献；使用“python3 script/validate_survey.py”进行验证。
7. 在 [../STATUS.md](#) 中跟踪进度。

一页成功检查：(1) 预测的智能编译器？ → §5.1/路线图。(2) 包括软硬件良好设计使能/拉起在内的四项智能体工作？ → §5.1/堆栈。(3) 证据与噪音？ → A 级证据 vs C。

路线图

北极星目标：智能体编译器** — 一个基于经典数据面（MLIR/LLVM/Inductor/Triton/Tile/vendor backends）的混合控制面（LLM/智能体+判定预测机），包括HW-SW软硬件良好设计循环。软件目录和芯片功能是该目标的证据，但其本身并不是目的。

同伴： [调查.md](#) §5 · [堆栈.md](#) · [指控.md](#) · [冲突.md](#)

地平线 A — 2027-2028（近）

以 C1-C10 为条件的可证伪草图。

什么船

能力	预测状态	主要证据	冲突
智能体可寻址编译器	工具 API、结构化 IR 摘要、准入/回退路径在 LLVM/Inductor/供应商工具链中变得正常	ACCLAIM、HintPilot、智能体编译、mlir 智能体（负自由重写）	C3、C6
在线专业化（工作a）	热内核/路径在 CI 中针对某些产品使用智能体或进化搜索（ACF、提示、Triton/Helion 优化）——尚未为所有构建提供静默默认设置	CompileIQ、GEAK、AutoKernel、Kernel Forge	C2、C5
离线启发式综合（作业b）	Magellan 级 C++ 启发式进化和 MLGO 神经顾问仍然有效（并行投注）	Magellan、EmitC-MLGO RFC	C1
工程智能体（工作c）	编译器-主机在严肃的 LLVM/AI 编译器组织中进行 PR 审查 (Alive2/opt)；通用锻造人工智能保持用户体验	Archer	C7
Bring-up / 软硬件良好设计智能体（工作d）	SIM + 芯片上覆盖范围优先的 ATen/Triton 后端生成成为新 ASIC 的标准	TritonX、KernelEvolve、升腾分级诊断	C9
DSL 表面	Triton 系列 (Triton/Helion) 仍然是主要的智能体训练表面；Tile/CuTe/HIP/FlyDSL发力多DSL技能	Helion、CompileIQ Helion 路径、TRT-LLM 智能体、KForge	C4

到 2028 年不发货的是什么

- 无约束的 LLM 端到端取代了“opt”/Inductor，无需经典承认（C6）。
- 适用于所有供应商的单一“一个智能体 IR”（C4 未解决）。

- 内核智能体在融合密集的公共阶梯上一致击败了 eager/libraries (C2)。
- 智能体设计硅微架构自主 (软硬件良好设计是反馈给人类/EDA, 而不是流片自动驾驶仪)-参见 Horizon B。

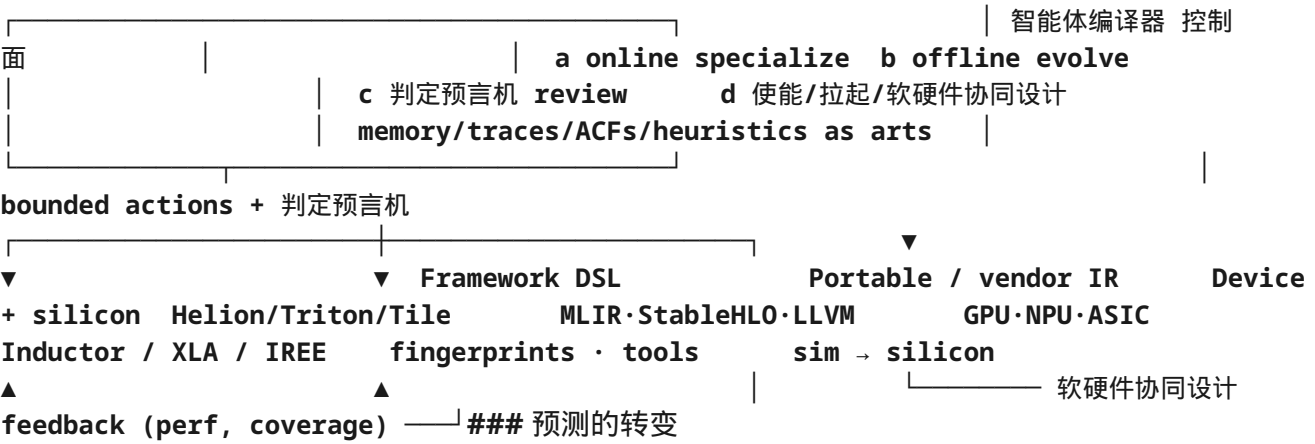
近期里程碑 (观看)

1. 公共 Magellan/OpenEvolve llvm 补丁或 EmitC-MLGO 默认值 (C1)。
2. CompileIQ/GEAK 发布 p50/p90 + 固定痕迹 (C2)。
3. TritonX 类似使能/拉起在 Meta 之外复制 (第二个 ASIC 供应商) (C9)。
4. PyTorch/LLVM 发行说明将智能体/ACF 作业列为支持的工作流程 (C5)。

地平线 B — ~2029-2031 (从 2026 年起接下来 ~5 年)

仍然以智能编译器为产品; 硬件软硬件良好设计是指产品如何吃掉 $O(\text{ops} \times \text{devices} \times \text{gens})$ 矩阵。

架构演变text



主题	5 年成果	信心
默认编译路径	经典降低仍然是默认的；智能体ic专业化是选择加入然后CI门控默认热路径	中高
文物商店	VCS 发展出一流的 ACF、进化启发式、经过验证的内核、优化内存、使能/拉起语料库	高
异构服务	智能体多后端内核/forge 循环是非 NVIDIA 设备保持可行的方式 (MTIA/AMD/Intel/NPU)	中高 (KernelEvolve、KForge、TritonX)
HW 软硬件良好设计循环	流片前：智能体 + sim 生成覆盖率和编译器压力；硅后：智能体图 ISA/IR 痛点 → RFC 为下一个芯片/方言。人类+ EDA 仍然拥有流片	中等
验证	本地正式 (Alive2级) +统计服务判定预测机+OpInfo级套件组成；GPU/NPU全方案正式尚未完成	中等
人类角色	专家拥有判定机、所有权、安全性、ISA合同；智能体草案内核/启发式/后端	高
仍然不会发生	完全自主的芯片+编译器联产，无需人类架构意图；端到端法学硕士作为“选择”	高

特定于代码设计的路线图（仍然以智能体ic编译器为中心）

相	智能编译器可以	硬件团队获得
预硅	QEMU/sim 上的 TritorX 式覆盖；ISA 文档草稿下的 KernelEvolve 式搜索	早期的“我们可以运行 NanoGPT/DLRM 操作吗？”信号; IR/方言错误
培养	覆盖智能体 → 性能智能体阶梯	ATen/Triton 后端骨架的周→小时
稳态	跨代在线专业化 + 记忆 (KernelBlaster)	L2/TMA/SRAM 规则更改时的可移植性
下一个流片	来自智能体跟踪的聚合故障模式（非法操作、对齐、缺失原子）	优先 ISA/内存系统/编译器传递请求

非目标：调查 EDA/RTL LLM 工具，除非它们将循环闭合到编译器 **IR**、内核或承认主预测机。

此路线图的成功指标

- 1. 新的 ASIC 能否公开智能体可寻址编译/测试 API 并在发布周期内通过智能体达到 >80% ATen 覆盖率？（TritorX吧）
- 2.智能体ic专业工作是否在CI（p50）中显示分布式胜利，而不仅仅是最好的内核？(C2)
- 2. Magellan 级和 MLGO 级路径是否仍有效或已解决？(C1)
- 3. 堆栈是否将跟踪/ACFs/启发式视为可供所有者审查的工件？（\$4.8–4.9）

当冲突解决或新的 A 级证据软硬件良好设计证据落地时进行更新。

软件堆栈和硬件软硬件良好设计 reshape

焦点：不是“一般的人工智能软件”，而是一个智能编译器如何改变从框架用户体验到芯片反馈的层。

同伴： [路线图.md](#) · [调查.md](#) §5 · [分类.md](#) · [冲突.md](#)

图层图（今天 → 智能体）

层	经典角色	通过智能编译器重塑	证据
1.模型/框架	torch.compile, JAX/TF 导出	智能体消耗图形/区域；阿姆达尔级热门行动；将内核写回 eager/compile 路径	AutoKernel、Kernel Forge、智能体编译
2.内核 DSL	CUDA / Triton / Helion / 平铺 / CuTe / HIP	DSL 变成智能体训练+搜索界面；Helion 提高了抽象性；所需的多 DSL 技能	Helion、GEAK、CompileIQ、KForge、TRT-LLM 智能体 PR
3.便携式红外	StableHLO、MLIR 方言	必须公开指纹、工具API、合法性；自由重写失败	mlir智能体、StableHLO、MLIR
4.编译器中/后	LLVM/XLA/Inductor/NVCC 通过	离线智能体进化启发式；在线智能体挑选通行证/提示/ACFs；MLGO 顾问坚持不懈	Magellan、MLGO、ACCLAIM、HintPilot、CompileIQ
5.预言机和分析器	单元测试、Alive2、NCU	成为接纳门禁+奖励；异质硬件所需的联合分析器 (MPP)	Archer、LLM-VeriOpt、KernelEvolve、升腾诊断
6. 工件/VCS	二进制文件、时间表	ACFs、进化的 C++、经过验证的内核、优化内存、使能/拉起语料库	CompileIQ、Magellan、KernelBlaster、TritorX
7.服务运行时	vLLM、TRT-LLM、自定义广告堆栈	智能体循环专门服务内核；不得破坏图形级选项	GEAK、KForge 与 TRT-LLM、KernelEvolve
8.硅/SIM	手册使能/拉起，ISA 文档	智能体在sim + 硅上生成后端；跟踪信息告知下一步 ISA/IR（软硬件良好设计）	TritorX、KernelEvolve、升腾NPU纸张

堆栈上的四个智能体作业text (a) Online specialize → layers 1-2-4-5-7 (CompileIQ, GEAK, AutoKernel, ACCLAIM) (b) Offline evolve → layer 4 (+ artifacts) (Magellan / AlphaEvolve) (c) Oracle engineering → layers 4-5-6 (Archer, CCC-adjacent) (d) Bring-up / 软硬件协同设计 → layers 2-5-8 (TritorX, KernelEvolve, Ascend diagnosis, KForge)工作(d)是HW-软硬件良好设计扩展：仍然是一个智能体编译器/工具链问题（内核、方言、测试），而不是通用芯片LLM设计。

堆栈重塑论文（声明 ID）

身份证	论文	状态
S1	控制平面变得智能化；数据面保持经典	支持 — 参见权利要求 A1
S2	可移植性从“一次写入 IR”转变为“智能体 + 每个后端的判定预示机”，而 IR 仍然是必要的基础	有争议 — C4、C8
S3	新的一流工件（ACF/启发式/内存/跟踪）改变 CI 和代码审查	支持 — A3
S4	定制 ASIC 的竞争力越来越依赖于智能体能/拉起延迟	支持（工业）— TritorX/KernelEvolve；观看第二供应商复制品 — C9
S5	分析器和编译器内部结构从人类 IDE 工具转向智能体 API	观看 — KernelEvolve MPP，提升层次结构

什么“不”与堆栈重塑混淆

相似	为什么本调查较弱
应用程序存储库上的通用编码智能体	Nocompile 主板机 → C 级证明
纯 EDA/RTL LLM，无内核/IR 循环	超出范围，除非与编译器承认相关
供应商 SKU 列表不含智能体/判定机 API	仅 B 级证明基线

阅读顺序

- 1. [路线图.md](#) 地平线 A/B
 - 2. 该文件的图层表
- 2. [指控.md](#) A/S/P*
- 3. 索引中 HW 软硬件良好设计 和 GPU 内核 下的摘要 ★

索赔地图

从可证伪的主张到文献摘要的活生生的地图。当证据改变状态时更新。

状态：支持 · 有争议 · 观看 · 伪造

架构（智能编译器）

身份证	索赔	状态	最好的证据	冲突
A1	智能体负责搜索/编排/合成；编译器负责降低、合法性、措施、回退路径	支持	ACCLAIM、智能体编译、HintPilot、mlir 智能体（负）	C3、C6
A2	四个智能体工作坚持：（a）在线，（b）离线启发式，（c）工程/审查，（d）使能/拉起/软硬件良好设计	支持	(a) CompileIQ/GEAK/AutoKernel; (b) Magellan; (c) Archer; (d) XPTECT27X/KernelEvolve	C5、C9
A3	ACFs、进化启发式、验证内核、优化内存、使能/拉起语料库成为一流神器	支持	CompileIQ、Magellan、KernelBlaster、TritonX	—
A4	默认值保持经典，直到智能体在 CI 中的分布上获胜为止有争议	供应商博客与 CompileIQ 2–3% 文档、KernelBench-X	C2	
A5	无约束LLM不会很快取代opt/Inductor	支持	mlir智能体；混合A级证据优势	C3、C6

进程和堆栈

身份证	索赔	状态	最好的证据	冲突
P1	到 2028 年，离线启 发式综合和 MLGO 神经顾问仍将保持并 行赌注	有争议	Magellan 与 EmitC- MLGO	C1
P2	多DSL/多供应商智能 体技能变得常态	观看	KForge、GEAK v3、TRT-LLM 智能 体、 Helion+CompileIQ	C4
P3	Compiler-判定机评 审击败 generic forge AI 选择 PRs	支持（方向）	Archer； C级证据降 级	C7
S1	Stack reshape 是传 统数据面的控制平面 智能体	支持	堆栈.md·A1	C6
S4	定制 ASIC TTM 越来 越多地受到智能体化 使能/拉起的控制支持 （工业）	TritorX、 KernelEvolve	C9	
S5	分析器/编译器内部成 为智能体 API	观看	KernelEvolve MPP，升腾分级诊断	C3

协同设计（仍然以智能体ic编译器为中心）

身份证	索赔	状态	最好的证据	冲突
H1	Pre-silicon sim + 智 能体在流片前提供编 译器/ISA 反馈	支持（早期）	TritorX QEMU 未来 设备	C9、C10
H2	覆盖范围第一的智能 体，然后是性能智能 体是使能/起步的阶梯	支持	TritorX → KernelEvolve	C9
H3	到 2031 年，智能体 不会自动流片芯片； 他们强调编译器/ISA	支持（预测）	TritorX/ KernelEvolve 的范 围（内核/工具链）	C10

结算观察

信号	移动	摘要
公共 Magellan llvm + OpenEvolve 配方	C1	麦哲伦, openevolve
Android/Chrome 上默认的 EmitC-MLGO	C1	mlgo-emitc-rfc
p50/p90 公共 ACF/内核跟踪	C2	编译 iq-*, kernelbench-x
第二个非 Meta ASIC 再现 TritonX 级覆盖范围	C9	三梅花
智能体 IR 合约, 免费重写胜过顾问	C3	acclaim 与hintpilot/智能体编译

本页记录了论文、供应商博客、OSS 存储库和论坛之间的分歧，这些分歧对于预测下一代人工智能编译器以及智能体如何改变未来至关重要。我们不强迫过早解决；每一次冲突都会说明双方的立场、为什么它对预测很重要，以及如何解决它。

同伴：[调查.md](#) § [未来](#) · [产品.md](#) · [REPOS.md](#)

如何读取冲突行

领域	意义
索赔 A / 索赔 B	主要来源的竞争读物
为什么重要	获胜者如何改变下一代编译器草图
结算信号	什么证据可以做出决定？

C1 — 进化可交付的 C++ 启发式算法与嵌入神经顾问程序 (Magellan 与 MLGO)

侧面	来源	职位
A — 综合可读的启发式	Magellan 纸张/幻灯片； AlphaEvolve 血统； OpenEvolve	LLVM/XLA 内部智能体重写 EVOLVE-BLOCK C++；船像人一样经过；Magellan 声称内联胜过数十年的手动工作；幻灯片希望超越NN政策
B — 保持/改善神经 MLGO	LLVM 话语 EmitC RFC； IR2Vec+MLGO RFC；正在进行的 MLGO 会议 (2026 年)	生产版 Chrome/Android/Fuchsia 仍然投资树内 NN 顾问；EmitC/TOSA 路径消除了 TF 构建依赖，因此神经策略保持可部署性

为什么重要。如果 A 获胜，下一代控制面是一个 离线进化编码智能体，其输出是普通编译器源代码。如果 B 获胜，数据面将保持学习策略作为一流的运行时顾问，并且智能体主要帮助训练/特征那些神经网络。

结算信号。公共 Magellan 启发式方法登陆 llvm-project 并在相同大小/性能的应用程序上取代 MLGO；或 EmitC-MLGO 成为 Android/Chrome 的默认路径，而 Magellan 保留在 Google 内部。

C2 — 供应商“生产智能体”战胜清醒的基准上限

侧面	来源	职位
A — 巨大的商业胜利	NVIDIA CompileIQ 博客（TritonBench/Helion 上的元数据高达约 15%）；AMD GEAk v3 博客（存储库级别 HIP/Triton/FlyDSL）；AlphaEvolve 云GA	智能体/自动调谐控制面已经实现了有意义的生产加速
B — 硬质天花板和回归	CompileIQ 文档（在高度优化的内核上通常为 2–3%）；KernelBench / KernelBench-X（许多正确的内核比 eager 慢；refine↑ 正确性可以 ↓avg 加速；融合困难）	标题加速是根据工作负载选择的；迭代智能体可能会以牺牲性能为代价来追求正确性

为什么重要。“智能体成为默认编译”的预测需要中位数/CI 获胜，而不仅仅是精心挑选的内核。过度声称会延迟对决策预示机和痕迹的投资（\$4.2–4.3）。

结算信号。可重现的公共 ACF/内核智能体跟踪，具有固定的编译器版本，报告 分布 (p50/p90) 不仅是最佳情况；KernelBench-X 风格的融合套件仍未解决或已得到解决。

C3 — 法学硕士可以自由重写 IR/代码，而不是必须保持咨询

侧面	来源	职位
A — 多级 LLM 重写可与测试配合使用	ACCLAIM（编译器-法学硕士合作）；GEAK 生成-评估-反映；内核智能体	用编译工具指导智能体交错LLM重写；测试/简介录取候选人；报告加速
B — 直接 IR 转换失败	mlir 智能体（IR 变换上的前沿模型低于身份）；HintPilot/智能体编译设计（仅限提示/模板）	无约束的 IR 重写是不安全/弱的；成功的系统限制行动空间

为什么重要。下一代架构要么公开广泛的重写API（具有强大的判定预测机），要么公开狭隘的咨询API（提示、旋钮ACFs、启发式块）。这些是不同的产品。

结算信号。共享智能体 IR 合约 + 判定预测机套件，其中免费重写在正确性×性能方面始终击败受限顾问；或对仅供咨询的接纳门控进行行业标准化。

C4 — 内核 DSL 未来：Triton 与 CUDA Tile（以及朋友）

侧面	来源	职位
A — Triton 仍然是智能机身表面	Inductor默认路径；保护10X；KernelLLM；GEAK Triton 路径；很棒的LLM驱动的内核生成目录	生态系统、基准测试和智能体已经融合在 Triton 上
B — 平铺/CuTe/HIP/FlyDSL 表面碎片	NVIDIA CUDA Tile + CompileIQ；TRT-LLM Claude 智能体适用于 CuTe/TileIR/Triton/CUDA；GEAK 多语言（HIP、FlyDSL、TileLang）	供应商推出硬件原生磁贴 IR；智能体必须成为多 DSL 或失去峰值

为什么重要。训练数据、工具 API 和“一个智能体 IR”赌注的成功或失败取决于此选择（§4.4、§4.7）。

解决信号。一个 DSL 成为主导的智能体训练语料库；或者便携式瓷砖 IR 获胜；或多 DSL 智能体（TRT-LLM 技能模式）成为常态。

C5 — 在线编译智能体与离线编译工程智能体

侧面	来源	职位
A — 在线（在编译/服务循环中）	CompileIQ ACFs；HintPilot；智能体编译；服务堆栈上的 GEAK；用于算法搜索的 AlphaEvolve 云	用户在最佳时间支付代币/GPU；工件是每个工作负载的配置/内核
B — 离线（更改编译器一次）	Magellan；MLGO 培训；Archer 公关审查；人类克劳德 C 编译器	智能体更改编译器源或审查 PR；用户随后会得到经典的 -O3/opt

为什么重要。这是两种不同的“智能体未来”。混合组织可能两者都需要，但路线图和成本模型不同。

结算信号。哪条路径在 12-24 个月的 PyTorch/LLVM/CUDA 发行说明中显示为“默认”标志或 CI 作业。

C6 — 智能体替换编译器与智能体是控制面

侧面	来源	职位
A — 智能体可以构建/替换大型编译器表面	Anthropic CCC（~100kLoC Rust 编译器）；HN/论坛的一些乐观情绪	智能体团队编写编译器；经典工程瓶颈缩小
B — 混合控制/数据面是持久的模式	新的编译器堆栈调查；mlir智能体限制；ACCLAIM配合取景；供应商堆栈仍然提供 TRT-LLM/Inductor/XLA	数据平面（降低、合法性、措施）保持经典；智能体搜索/合成/建议

为什么重要。我们的调查执行结论押注于 B。A 会将目标重写为“智能体编写的编译器”作为主要目标。

解决信号。一个生产人工智能堆栈，其默认下降路径是智能体生成的，没有经典的承认/回退路径编译器，而不是研究演示。

C7 — 通用 SCM AI 审查 vs 编译器-判定预测机审查

侧面	来源	职位
A — Forge AI 就足够了	Gerrit ai-code-review / ReviewAI / 原生 AI 聊天；通用 GitHub PR 机器人	把LLM放在diff上；规模人工审核
B — 需要编译器专用工具	Archer (Alive2/LLUBI/opt); LLVM Discourse智能体-公关体验	Miscompiles need domain 判定机；一般审查仅限于 HITL UX

为什么它对这个调查很重要。通用 Gerrit 插件对于下一代编译器来说是薄弱的证据；Archer 级工具功能强大。对没有判定预测机的 forge 插件进行编目与预测目标不一致。

结算信号。阻止合并失败的 Alive2/KernelBench 类检查的 Gerrit/GitHub 机器人成为 llvm-project 或主要 AI 编译器中的默认设置。

C8 —“AI 编译器”是指 DL 图形编译器与 LLM-for-LLVM

侧面	来源	职位
A — AI 模型编译器	TVM/XLA/Inductor/TRT-LLM/ OpenVINO 产品文档	下一代=更好的图→设备堆栈 (Tile、StableHLO、Neuron NKI)
B — 编译器人工智能	Magellan、LLM 编译器、Compiler-R1、Archer	下一代 = LLVM/XLA/内核工程中的智能体

为什么重要。商业目录超重 A；研究目录超重 B. 预测必须保持两个堆栈收敛，以智能体作为横切点，而不是选择一个目录。

结算信号。（此处已部分说明。）在 DL 编译器（CompileIQ、GEAK、Magellan→XLA）上公开智能体 API 的产品是收敛证明。

C9 — 覆盖优先的使能/拉起智能体 vs 峰值性能内核智能体

侧面	来源	职位
A — 覆盖范围解锁设备	TritonX (481 ATen 操作、OpInfo、MTIA sim+硅)；Intel Arc 上的 KForge	新的 ASIC 在峰值内核之前需要任何正确的后端；智能体应首先最大化运营商覆盖范围
B — Perf 智能体是产品	KernelEvolve、GEAK、AutoKernel、KernelBench fast_p	与 eager/library 相比，如果没有加速，智能体编译不会支付 TCO；仅覆盖后端仍然输给 NVIDIA 堆栈

为什么重要。智能体ic编译器路线图需要一个阶梯（覆盖范围→性能）而不是单一目标。仅优化 HotGEMM 的协同设计程序将导致模型搁浅；只追逐 OpInfo 的程序永远不会赢得服务。

解决信号。公共剧本，在同一 ASIC 上对 TritonX 级覆盖范围进行排序，然后对 KernelEvolve 级性能进行排序，并提供服务指标 - 或者一个目标主导全行业的发布标准。

C10 — 智能体编译器软硬件良好设计反馈与自主芯片设计

侧面	来源	职位
A — 智能体软硬件良好设计硅	广泛的“芯片设计人工智能”叙述； sim 的乐观态度 使能/拉起	法学硕士将提出 ISA/microarch 以及端到端循环编译器
B — 智能体压力工具链；人类/EDA 自己的流片	TritonX/KernelEvolve 实际范围 (内核、方言、测试、分析器)；本 次调查的路线图	智能体编译器缩短 SW TTM 并归档 ISA/IR 疼痛报告；自主流片是一个不 同的领域

为什么重要。 将调查重点放在目标智能编译器上。 仅当 HW 通过 kernels/IR/判定结果机关闭循环时，HW 才在范围内。

结算信号。 一种生产芯片，其微架构主要是智能体提出的并通过智能体ic编译进行验证 - 不仅仅是智能体编写的没有编译器认可的RTL片段。

本次调查的工作立场（直至结算）

1. 首选混合对照/数据面 (C6-B) 作为预测基线。 2.将Magellan式启发式综合和MLGO 神经顾问视为并行生产赌注（C1未解决）。
2. 折扣单号供应商加速，无需分配/判定预测机上下文 (C2)。
3. 假设约束动作+强判定预测机，直到自由重写证明自己为止（C3）。
4. 将通用 SCM AI 插件降级为 C 级证据 (C7)。
5. 将 DL 编译器产品保留为 B 级证据基线，而不是下一代 (C8) 的定义。
6. 通过覆盖率→**perf**智能体阶梯在sim+silicon上进行代码设计（C9）； 不要扩展到自主 EDA (C10-B)。

当冲突得到决定性的公开解决时更新此文件。

系统比较

代表性系统的快照。数字根据作者报告；跨基准比较不是同类比较。在选择架构时，更喜欢机制而不是总体加速。

系统	层	智能体形	正确性门	标题	地点/来源
Meta LLM 编译器	LLVM 红外/asm	基础模型	编译器应用通行证	自动调整大小潜力的约 77%	CC 2025
Compiler-R1	LLVM通行证	单一智能体+工具+强化学习	opt + IR 指令计数	与 -Oz 平均值相比, IR 尺寸约为 8.5%	NeurIPS 2025
LLM-VeriOpt	LLVM 红外窥视孔	训练有素的小模特	Alive2 等效项	~90% 可验证正确	CGO 2026
Magellan	启发式 (C++)	智能体编码 + AlphaEvolve	编译+宏平台	击败数十年的内联启发法	C4ML@CGO'26 / LLVM 开发管理
Aware编译器	传球顺序	多回合智能体-env	编译/运行奖励	知识桥梁功能↔通行证	arXiv 2025
AutoPass	通行证/旗帜	多智能体, 仅限推理	编译+剖析证据	实用预算, 无需离线RL	arXiv 2026
HintPilot	源编译指示	迭代 RAG 细化	编译器验证提示+测试	与 -Ofast 相比高达 6.88× 地理平均值	ACL 调查结果 2026
ACCLAIM	C/asm多级	指南+等级+测试智能体	LLM测试+编译	比单独编译器高出 1.25 倍	arXiv 2026
推理编译器	TVM 时间表	LLM 提案 + MCTS	TVM 编译/测量	样本效率与 MetaSchedule	NeurIPS 2025
智能体编译	CUDA 推理	有限搜索顾问	CUDA 模板+检查+工作台	~4–5.7× vs eager (小型法学硕士)	arXiv 2026
GEAK	Triton / AMD GPU	生成/评估/反映/优化	单元测试+计时	执行速度高达 63%; ~2.59×	AMD/arXiv 2025
KernelLLM	PyTorch → Triton	专业8B型号	KernelBench-Triton	强 Pass@k 与规模更大的普通法学硕士	元高频 2025
MLIR智能体	MLIR / LLVM	使用工具的智能体(MCP)	传球追踪+长凳	法学硕士不擅长直接 IR 重写	UCB 酒吧
生成编译	Rust 前端	解码中反馈	密封剂 + rustc	编译器在生成期间处于活动状态	arXiv 2026
下一个	编译器FMware / 意图	SE 3.0愿景	多目标质量门	编译提示/智能体/参数	arXiv 2025
NVIDIA CompileIQ	NVCC/PTXAS 旋钮	进化搜索	测量真实内核	热 Triton/CUTLASS 内核上≤15%	CUDA 13.3 博客
MLGO	LLVM 启发式	树内强化学习策略	编译器语义	产品内联/regalloc	谷歌/LLVM
KernelBench	评估安全带	不适用 (基准)	正确 + fast_p	边境<20% one-shot typical	ICML 2025

系统	层	智能体形	正确性门	标题	地点/来源
TritorX	PyTorch ATen / Triton-MTIA	FSM 智能体 使能/拉起	OpInfo + model ops	481 ops; ~84% pass; sim+silicon	MLSys 2026
KernelEvolve	Triton (+TLX) multi-HW	Graph search + HW RAG	TritonBench + federated profilers	Hetero NVIDIA/AMD/MTIA prod	Meta 2025/26
KForge	Multi-DSL / multi-vendor	Gen ↔ perf-analysis 智能体	Compile + correct + profile	+2.12% vs TRT-LLM; 5.13× on Arc L2	arXiv 2026
AutoKernel	Triton/CUDA on PyTorch models	Keep/revert 智能体 loop	5-stage harness	Beats eager & torch.compile on hot ops	arXiv 2026
Ascend hierarchical diagnosis	Triton-NPU	Escalating compiler-grounded 智能体	Profile → IR → compiler source	4.35× geo-mean on 37 ops	arXiv 2026
Helion	PyTorch → Triton DSL	Autotune (not LLM)	Config search	Geomean >在报告的套件上编译/Triton	PyTorch 2025

在线与离线智能体

模式	当它运行时	典型神器	示例
在线	在程序或模型的编译/专门化时	通过列表、提示、内核选择、ACF 旋钮	HintPilot、智能体编译、CompileIQ、Compiler-R1 推理
离线	编译器工程/模型训练	C++ 启发式、基础权重、数据集	Magellan、Meta LLM 编译器培训、MLGO 培训

相关工程实验（相邻）

工作	为什么它对这项调查很重要？
人类克劳德 C 编译器	作为编译器编写者的智能体；安全带+测试占主导地位
LLVM 智能体公关评论 讨论帖	编译器专用工具 >> 用于 opt 审查的通用 SWE 智能体

“AI编译器”的两种含义

感觉	意义	示例
人工智能编译器	将神经图降低为加速器的系统	TVM、XLA/OpenXLA、MLIR 方言、TorchInductor → Triton、IREE、TensorRT
编译器人工智能	选择通道、重写 IR、编写启发式/内核的法学硕士/智能体	Meta LLM 编译器、Compiler-R1、Magellan、GEAK、HintPilot

这项调查将下一代视为它们的合并：控制面上的智能体，数据面上的编译器。

LLM 角色分类（新编译器堆栈，2026）

来自新编译器堆栈：法学硕士和编译器协同作用的调查 (arXiv:2601.02045):

- 1. 选择器 — 在预定义的编译器操作或候选者中进行选择（优化遍历序列、调度移动、CUDA 模板系列）。
- 2. 翻译器 — 重写源代码/IR/汇编（除非门控，否则正确性风险最高）。
- 3. 生成器 — 综合新的编译器工件（C++ 启发式、内核、工具、数据集）。

最强大的 2025-26 系统是选择器或生成器包含在混合验证中，而不是单独的自由格式翻译器。

编译循环中的智能体角色

角色	工作	示例
顾问/选择者	对候选人进行排名、标记区域、建议通过	智能体编译、Meta LLM 编译器、AutoPass
翻译/重写	源代码/IR/asm 重写或提示插入	ACCLAIM、HintPilot、LLM-VeriOpt
神器生成器	启发式、内核、MCP 工具	Magellan、GEAK、mlir 智能体
协调员	预算、IR 级别、停止条件	ACCLAIM引导智能体，GEAK引导
测试员/评论家	测试、Alive2、配置文件、完善提示	ACCLAIM 测试智能体，生成编译
寻找合作伙伴	为 MCTS/evolution 提出节点建议	推理编译器，AlphaEvolve
培育/软硬件良好设计	覆盖范围→SIM+硅上的性能；ISA/IR 反馈	TritorX、KernelEvolve、奥升德诊断、KForge

经典AI编译器栈（底层）
text Framework capture torch.compile / Dynamo, JAX, TF
graphs ↓ Portable HLO StableHLO (TF/JAX/PyTorch ↔ XLA/IREE) ↓
MLIR dialects multi-level lowers, rewrites, vendor dialects ↓ Schedule /
kernels TVM MetaSchedule, Inductor→Triton, CUDA Tile IR, CUTLASS ↓ Serving
runtime vLLM, FlashAttention, CUDA Graphs, decode paths ↓ CPU / legacy
IR LLVM opt pipelines, PGO / AutoFDO, MLGO advisors## 规范混合循环text Capture →
Analyze regions → 智能体 proposes → Compiler checks & lowers → Verify /
test (empirical or formal) → Benchmark / select → Feedback to
orchestrator → Fallback if unprofitable不变：LLM输出指导搜索；他们不应该默默地定义未经检查的可执行行为。

出版物索引

出版物索引

每个搜索源一篇文献摘要。文件位于该索引旁边。

年份	亲切	集团	摘要	来源
2026	2026纸	调查与愿景	新的编译器堆栈：法学硕士和编译器良好作用的调查	来源
2025	2025纸	调查与愿景	Compiler.next：基于搜索的编译器，为软件工程的人工智能的未来提供动力	来源
2026	2026纸	调查与愿景	从形式理论的角度解读MLIR中的AI模型编译	来源
2018	纸	经典深度学习编译器	TVM：用于深度学习的自动化端到端优化编译器	来源
2020	纸	经典深度学习编译器	Ansoor：为深度学习生成高性能张量程序	来源
2021	公司	经典深度学习编译器	TVM 博客：自动调度器简介 (Ansoor)	来源
2020	纸	经典深度学习编译器	FlexTensor：自动调度探索和优化框架	来源
2022	2022纸	经典深度学习编译器	TensorIR：自动张量化程序优化的抽象	来源
2021	纸	经典深度学习编译器	MLIR：终结摩尔校正的编译器基础设施	来源
2025	2025公司	经典深度学习编译器	OpenXLA StableHLO 路线图	来源
2021	纸	MLGO 和 RL 健身房	MLGO：机器学习引导编译器优化框架	来源
2022	2022公司	MLGO 和 RL 健身房	谷歌研究博客：MLGO	来源
2022	2022公司	MLGO 和 RL 健身房	InfoQ：MLGO 框架将机器学习导入编译器优化	来源
2022+	代码	MLGO 和 RL 健身房	LLVM 文档：机器学习引导优化 (MLGO)	来源
2022	2022代码	MLGO 和 RL 健身房	CompilerGym (元/公平)	来源
2022	2022论坛	MLGO 和 RL 健身房	LLVM 讨论：ML 引导的编译器优化流程排序 (GSoC)	来源
2023	纸	编译器基础法学硕士	用于编译器优化的大型语言模型	来源

年份	亲切	集团	摘要	来源
2024	2024纸	编译器基础法学硕士	超大型语言模型编译器：编译器优化的基础模型	来源
2024	2024公司	编译器基础法学硕士	Meta AI 出版页面：LLM 编译器	来源
2024	2024公司	编译器基础法学硕士	Chris Cummins：法学编译器基础模型帖	来源
2024	2024纸	编译器基础法学硕士	大型语言模型生成反馈编译器	来源
2023	论坛	编译器基础法学硕士	HN：用于编译器优化的大型语言模型	来源
2024	2024论坛	编译器基础法学硕士	HN：Meta LLM编译器（长线程）	来源
2024	2024论坛	编译器基础法学硕士	HN：元大型语言模型编译器	来源
2025	2025纸	智能体和强化学习编译器	Compiler-R1：通过强化学习实现智能编译器自动调优	来源
2025	2025代码	智能体和强化学习编译器	Mind4Compiler/Compiler-R1（代码）	来源
2026	2026纸	智能体和强化学习编译器	LLM-VeriOpt：用于基于LLM的编译器优化的验证引导RL	来源
2026	2026纸	智能体和强化学习编译器	Magellan：使用AlphaEvolve 自主发现创新的编译器优化指令式 ★ 级证据离线作业	来源
2025	2025谈话	智能体和强化学习编译器	LLVM 开发者幻灯片会议：Magellan ★ 未来信号 (\$5.4)	来源
2025	2025纸	智能体和强化学习编译器	AlphaEvolve：用于科学和算法发现的编码智能体	来源
2026	2026公司	智能体和强化学习编译器	DeepMind 博客：AlphaEvolve 的影响	来源
2025	2025纸	智能体和强化学习编译器	AwareCompiler：智能体集成编译器优化	来源
2026	2026纸	智能体和强化学习编译器		来源

年份	亲切	集团	摘要	来源
			AutoPass : 用于编译器性能调优的证据引导LLM智能体	
2026	2026纸	智能体和强化学习编译器	HintPilot : 用于代码优化的基于LLM的编译器提示综合	来源
2026	2026纸	智能体和强化学习编译器	通过编译器-LLM 合作进行智能体代码优化 (ACCLAIM) ★ A 级证据 Q2/Q3	来源
2026	2026代码	智能体和强化学习编译器	亚马逊科学/赞誉 (GitHub) ★ A 级证据	来源
2026	2026纸	智能体和强化学习编译器	生成编译: 人工智能生成代码时的动态编译器反馈	来源
2025	2025纸	GPU 内核和推理编译器	KernelBench : 法学硕士可以编写高效的GPU内核吗?	来源
2025	2025公司	GPU 内核和推理编译器	斯坦福大学博客: KernelBench	来源
2025	2025代码	GPU 内核和推理编译器	扩展智能/ KernelBench	来源
2026	2026纸	GPU 内核和推理编译器	KernelBench-X : 用于评估LLM生成的GPU内核的综合基准	来源
2025	2025纸	GPU 内核和推理编译器	GEAK : 介绍 Triton 内核 AI 智能体和评估基准	来源
2025	2025公司	GPU 内核和推理编译器	AMD ROCm 博客: Triton 内核 AI / GEAK	来源
2025	2025代码	GPU 内核和推理编译器	AMD-AGI/GEAK 智能体	来源
2025	2025公司	GPU 内核和推理编译器	Meta KernelLLM (抱脸)	来源
2025	2025纸	GPU 内核和推理编译器	推理编译器: LLM引导的模型服务优化	来源
2026	2026纸	GPU 内核和推理编译器	智能体编译: 用于直接CUDA推理法学硕士指导编译器	来源
2025	2025代码		ucb-bar/mlir 智能体	来源

年份	亲切	集团	摘要	来源
		GPU 内核和推理编译器		
2025	2025公司	公司基础设施	NVIDIA: 关注您的算法 — CUDA Tile 处理硬件	来源
2026	2026公司	公司基础设施	NVIDIA: 使用 CUDA Tile 使用 C++ 开发性能 GPU 内核	来源
2026	2026公司	公司基础设施	NVIDIA CUDA 13.3: 平铺 C++ + CompileIQ	来源
2026	2026公司	公司基础设施	NVIDIA: 利用 CompileIQ提高内核性能	来源
2026	2026代码	公司基础设施	CompileIQ 文档	来源
2025	2025公司	公司基础设施	典型: MLIR 编译器基础设施怎么样?	来源
2026	2026公司	公司基础设施	Anthropic: 与毛茸茸的 Claudes 团队一起构建 C 编译器	来源
2026	2026公司	公司基础设施	Ars Technica: 十六岁的克劳德智能体创造了一个 C 编译器	来源
2026	2026公司	公司基础设施	Modular/Lattner: Claude C 编译器 — 软件的未来	来源
2026	2026论坛	公司基础设施	HN: 我们委托 Opus 4.6 智能体团队构建一个 C 编译器	来源
2025	2025论坛	论坛和研讨会	LLVM 讨论: LLVM ♥ ML 研讨会 2025 年	来源
2026	2026论坛	论坛和研讨会	LLVM 讨论: LLVM ♥ ML 研讨会 2026 CFP	来源
2023	论坛	论坛和研讨会	LLVM 演讲: ML 引导编译器优化研讨会 2023	来源
2026	2026论坛	论坛和研讨会	LLVM话语: 使用智能体进行自动审查 (约30个错误/207个 PR)	来源
2026	2026论坛	论坛和研讨会		来源

年份	亲切	集团	摘要	来源
			comp.compilers: Magellan 纸质通知	
2025	2025公司	论坛和研讨会	IEEE Pulse: 编译器优化中的法学硕士 — 挑战和未来方向	来源
2026	2026公司	论坛和研讨会	月光文献综述: Magellan	来源
2010 年代+	代码	正确的血统	Souper: LLVM IR 的超级优化器	来源

2026 | 2026纸 | 源代码控制和审查智能体 | [Archer: 面向编译器优化的智能体审查](#) | 来源 |

2026 | 2026代码 | 源代码控制和审查智能体 | [cuhk-s3/Archer \(GitHub\)](#) | 来源 |

2024+ | 代码 | 源代码控制和审查智能体 | [Gerrit 插件: ai-code-review \(googlesource\)](#) | 来源 |

2025+ | 代码 | 源代码控制和审查智能体 | [amarula/评论ai-gerrit-插件](#) | 来源 |

2025+ | 代码 | 源代码控制和审查智能体 | [GerritForge/ai-review-智能体-provider](#) | 来源 |

2025 | 2025代码 | 源代码控制和审查智能体 | [OpenEvolve \(AlphaEvolve型OSS\)](#) | 来源 |

2025 | 2025代码 | 源代码控制和审查智能体 | [HeuriGym \(cornell-zhang/heurigym\)](#) | 来源 |

2025 | 2025代码 | 源代码控制和审查智能体 | [meta-pytorch/内核智能体](#) | 来源 |

2026 | 2026代码 | 源代码控制和审查智能体 | [NVIDIA/CompileIQ \(GitHub\)](#) | 来源 |

2026 | 2026代码 | 源代码控制和审查智能体 | [人类学/claude-c-编译器](#) | 来源 |

2021+ | 代码 | 源代码控制和审查智能体 | [谷歌/ml-编译器-opt](#) | 来源 |

2026 | 2026代码 | 源代码控制和审查智能体 | [ZJU-PL/hintpilot](#) | 来源 |

| 正在进行 | 代码 | 源代码控制和审查智能体 | [llvm/llvm-project \(主机存储库\)](#) | 来源 |

2026 | 2026纸 | 调查与愿景 | [迈向法学硕士时代的自动内核生成](#) | 来源 |

2025+ | 代码 | GPU 内核和推理编译器 | [flagos-ai/awesome-LLM 驱动的内核生成](#) | 来源 |

2026 | 2026公司 | 商业产品和建议 | [AMD ROCm: GEAK v3 内核优化智能体](#) | 来源 |

2025 | 2025论坛 | 论坛和研讨会 | [LLVM 讨论 RFC: EmitC 对 MLGO 的支持](#) | 来源 |

2026 | 2026代码 | 商业产品和建议 | [TensorRT-LLM PR: Claude 内核和编译的智能体/技能](#) | 来源 |

2026 | 2026公司 | 商业产品和建议 | [CompileIQ文档: 预期收益 \(2%到3%高度优化\)](#) | 来源 |

2025/26 | 纸 | 硬件软硬件良好的设计和加速器使能/启动 | [ML ASICs \(TritonX\) 的智能体式算子生成](#) ★ 使能/拉起/软硬件良好设计 | 来源 |

2025/26 | 纸 | 硬件软硬件良好的设计和加速器使能/启动 | [KernelEvolve: 最强加速器的智能体内核编码](#) ★ 多硬件产品 | 来源 |

2026 | 2026纸 | 硬件软硬件良好的设计和加速器使能/启动 | [NPU 上 Triton 的基于编译器的分层诊断](#) ★ Ascend | 来源 |

2026 | 2026纸 | 硬件软硬件良好的设计和加速器使能/启动 | [KForge: AI加速器的跨平台内核生成](#) ★ 多DSL | 来源 |

2026 | 2026纸 | GPU 内核和推理编译器 | [AutoKernel: 自主GPU内核优化](#) ★ Amdahl 智能体循环 | 来源 |

2026 | 2026代码 | GPU 内核和推理编译器 | [RightNow-AI/autokernel \(GitHub\)](#) ★ | 来源 |

2026 | 2026纸 | GPU 内核和推理编译器 | [Kernel Forge: CUDA 内核选项的智能体线束](#) | 来源 |

2026 | 2026纸 | GPU 内核和推理编译器 | [KernelBlaster: 用于CUDA的内存增强型上下文强化学习](#) | 来源 |

2025 | 2025公司 | 经典深度学习编译器 | [Helion: 用于便携式ML内核的高级DSL](#) | [来源](#) |
2025+ | 代码 | 经典深度学习编译器 | [pytorch/helion \(GitHub\)](#) | [来源](#) |

总计: 95 文献摘要

种类: ‘论文’·‘公司’·‘论坛’·‘谈话’·‘代码’

★ = 下一代 预测 的高信号 (更新 [文档/调查.md](#) §5 / [文档/ROADMAP.md](#) 时首选这些)。

另请参阅: [文档/REPOS.md](#) / [文档/产品.md](#) 作为 **A** 级证据/**B/C** 证据 (不是目录); [文档/冲突.md](#) 当来源不一致时; [文档/STACK.md](#)用于软件+硬件重塑。

导出笔记

- 完整的文献摘要保留在“publications/*.md”中 (未内联)。
- 层级映射: docs/REPOS.md、 docs/PRODUCTS.md。
- 重建: python3publish/build_pdf.py (构建 en / zh-CN / zh-TW)。